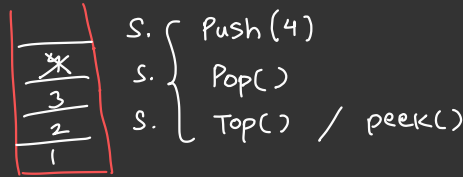


Stacks

- Revisit (Stacks)
- Problems (Easy - Medium)

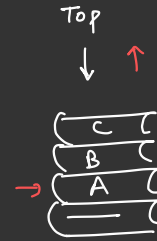
11/6



[Stack
Behaviour]

Stack s = new Stack(),

Push, Pop

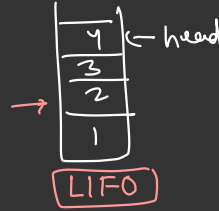
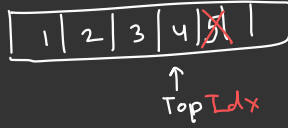


Stack

Stack
as
data structure
(explicit)

Problem solving

Call
Stack
(implicit)



array vs stack (limited
get(i) fn)

class Stack {

- array [MAX size] ~~X~~
- topIdx
- MaxSize

• push() { --- }

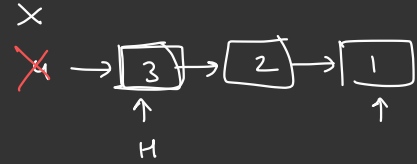
• pop() { --- }

• top() { --- }

empty() { --- }

full() { --- }

3



class Stack {

Node head

Node tail

• push() { --- } O(1)

• pop() { --- } O(1)

• top() { --- }

empty() { --- }

3

Problem #1

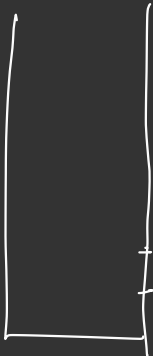
Paranthes Balanced

input = "((1+2) * 3 + (4-5)/6)" ✓

((1+2) * 3 + (4-6)) ✗

((1+2)) + (3-6/2) ✗

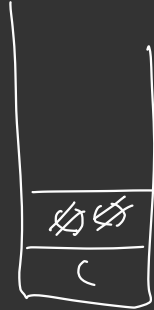
Ex-1
 ((1+2) * 3 + (4-5)/6)
 ↑ ↑ ↑ ↑ ↑ ↑



Stack is empty() → ✓

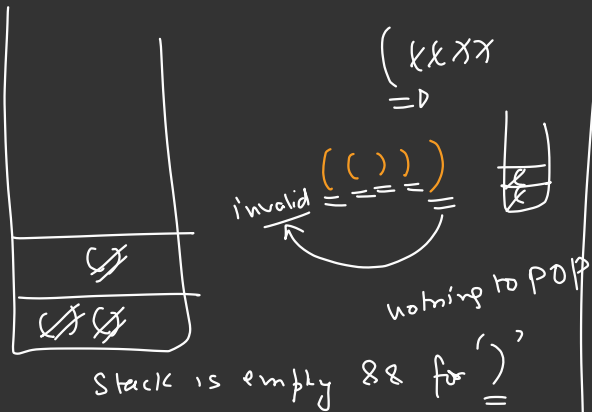
Ex-2

((1+2) * 3 + (4-6))
 ↑ ↑ ↑



⇒ Stack is not empty() → ✗

$$((1+2)) + (3 - 6/2)$$



↓
return false

The diagram consists of several hand-drawn elements:

- On the left, the words "To Do" are written in a stylized, cursive font and underlined twice.
- To the right of "To Do" is a vertical line, followed by the word "How" written in a similar cursive font and circled.
- Below "How" is a large green shape resembling a thick, curved arrow pointing downwards.
- To the right of the green shape is a blue shape resembling a thick, curved arrow pointing to the right.
- To the right of the blue shape is a brown shape resembling a thick, curved arrow pointing to the right.
- Below the blue and brown shapes are two horizontal lines, one under each shape.
- At the bottom center, there is a small upward-pointing arrow.

A diagram showing a complex shape on the left, composed of a brown 'C' shape and a blue '3' shape, with an equals sign and an upward arrow below it. This is followed by a plus sign and three individual shapes on the right: a brown 'C' shape, a brown '3' shape, and a green '3' shape, each with an upward arrow below it.

✓
True

$\left[\begin{array}{c} \{ \\ \} \end{array} \right]$
↑ No

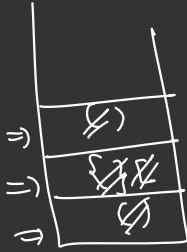
```
Stack s = new Stack();
```

```

=>
for (char x : input) {
    if (x == '(') {
        s.push(x);  → O(1)
    }
    else if (x == ')') {
        if (s.empty()) { return false; }
        else {
            s.pop();  → O(1)
        }
    }
}

```

return s.empty(), $\bullet$ T/F



Need to check Top for Matching
opposite pair before
popping

Interviews → reverse LL(Node head) {
 |||||
 }

func() {
 |||||
 }
 Stack s = new Stack().

Q Given an array find out the nearest smaller no on the left for every element - store -1 if no such no is found

arr = [1 6 4 12 3 8]
 output = [-1 1 1 4 1 3]

arr = [2 5 10 6 3 4 1]
 output = [-1 2 5 5 2 3 -1]

5 mins
 ↑
 think
 time
 =

Algo - Brute Force

 $O(N^2)$

for every $a(i)$

→ $\int$ Traversing i-1 to 0
find out a smaller element?

$O(N^2)$
time

$O(1)$
space

Algo - 2

$$a_{vv} = \begin{bmatrix} 2 & 5 & 10 & 6 & 3 & 4 & 1 \end{bmatrix}$$

output[] = [1 2 5 5 2 3 1]



6 → Remove
all No's
bigger
than 6

1 $\rightarrow$ Remove
all no
bigger
than 1

CODE

Stack s = new Stack();

Removing all
elements bigger
than
current element

for (i=0 _____ n-1) {
 x = arr[i]

 while (!s.empty() && s.peek() > x) {
 s.pop();
 }

 if (s.empty()) { output[i] = -1; }

 else { output[i] = s.peek(); }

s.push(x);

}

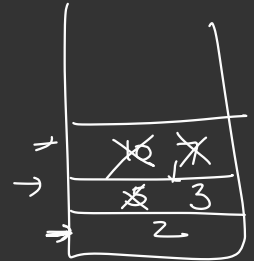
10:25



Break ->

DRY-RUN

⇒ 2, 5, 10, 7, 3, 4
 -1, 2, 5, 5, 2, 3



⇒ Pushed once

⇒ Popped once

$O(2N)$

$= O(N)$

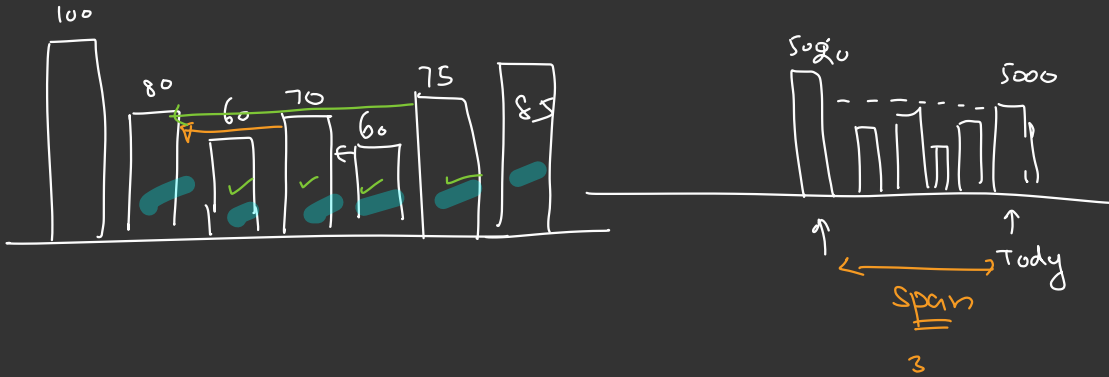
$$\Rightarrow \left\{ \frac{2}{-1}, \frac{5}{2}, \frac{10}{5}, \frac{7}{5}, \frac{11}{7}, \frac{4}{2} \right\} \quad \odot$$

$$\Rightarrow \left\{ \begin{array}{|c|c|} \hline \frac{10}{5} & \frac{7}{5} \\ \hline \frac{5}{2} & 4 \\ \hline \frac{2}{1} & 1 \\ \hline \end{array} \right\}$$

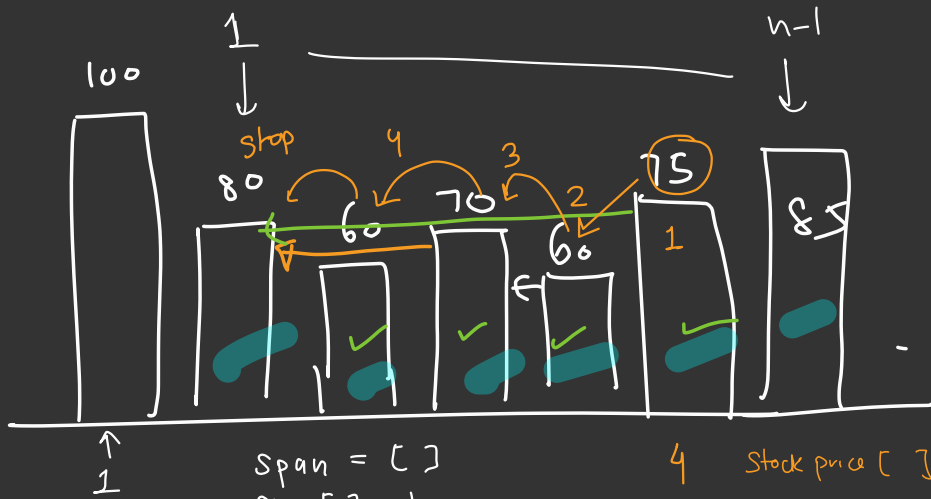
Stock Span Problem

→ Gold hits a '3 months' high

→ Sensex a '4 month' low



Input $\left\{ \begin{array}{l} \text{Stock price []} = \left\{ \begin{array}{ccccccc} D1 & D2 & D3 & D4 & D5 & D6 & D7 \\ 100 & 80 & 60 & 70 & 60 & 75 & 85 \end{array} \right\} \\ \text{Span []} = \left\{ \begin{array}{ccccccc} 1 & 1 & 1 & 2 & 1 & 4 & 6 \end{array} \right\} \end{array} \right.$



span = []
span[0] = 1

4 Stock price [] =
span [] =

for ($i=1$ to $n-1$) {

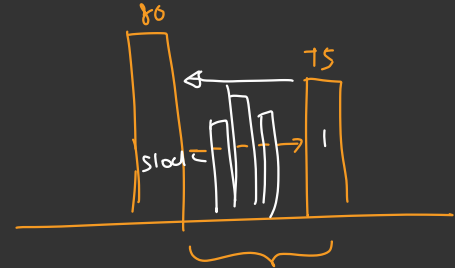
span[i] = 1

for ($j=i-1$, $j \geq 0$; $j--$) {
if (stock[j] > stock[i]) { break; }

span[i] = span[i] + 1

}

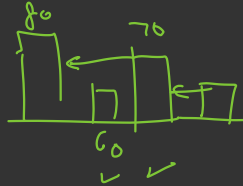
$O(N^2)$



4 days
{ 100, 80, 60, 70, 60, 75, 85 }
{ 1, 1, 1, 2, 1, 4, 6 }

100 > 80
80 > 70

60 > 70



idx

Stock price [] = { 100, 80, 60, 70, 60, 75, 85 }

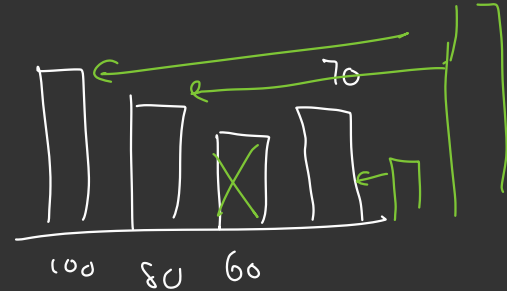
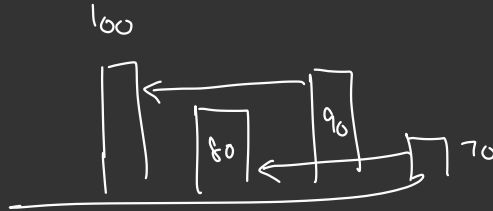
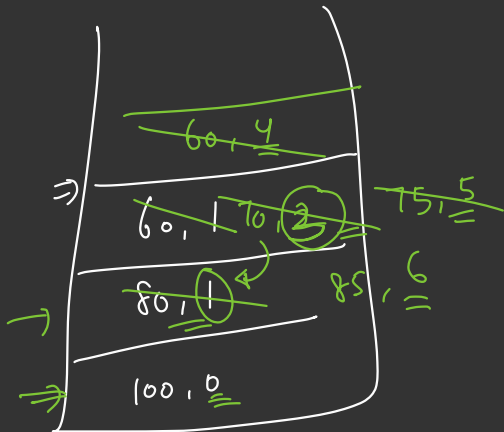
Span [] = { 2, 1, 1, 1, 2, 1, 4 }

↑
[prev larger]

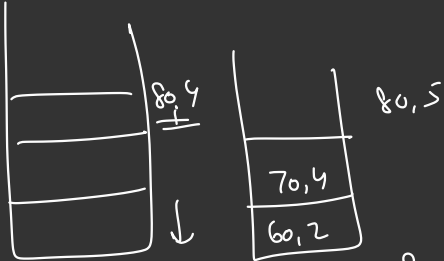
↑
i
(current) ⇒ 0 — (n-1)

Stack
store idx, val

Span ⇒ [(1) — (5)] = (4)



key idea - Pop smaller elements after seeing big element



Stack < int 7 s = new Stack<int>(),

span[0] = 1

s.push(0); // idx

for (i = 1 ——— n-1) {

current = stack(i)

while (!s.empty() && current >= stack[s.peek()]) {

s.pop(),

↳
span[i] = i+1 if s.empty()
= i - s.peek() if s is not empty()
s.push(i),

Code

```
Stack < int 7 s = new Stack<int>();
```

```
span[0] = 1
```

```
s.push(0); // idx
```

```
for (i=1 to n-1) {
```

```
    current = stock(i)
```

```
    while (!s.empty() && current >= stock[s.peek()]) {
```

```
        s.pop();
```

```
        span[i] = i+1 if s.empty()
                  = i - s.peek() if s is not empty()
```

```
        s.push(i);
```

}

$O(N)$ TC

$O(N)$ SC

idx	0	1	2	3	4	5	6	7
stock	2	5	7	3	6	8	4	4
span	1	2	3	1	2	6	7	1

stack

4, 7
9, 6
8, 5
6, 4
3, 3
1, 2

Stack[s.peek()]

775

3